

Generics

Java allows generic classes to use something called a **generic type**

Added in 1.5, it allows code to be flexible and also type safe

Old version of a list:

```
public interface OldStyleList {  
    public boolean add(Object obj);  
    public Object get(int index);  
}
```

New version of a list:

```
public interface List<T> {  
    public boolean add(T obj);  
    public T get(int index);  
}
```

Generics

Generics allows `List<String>`, `List<Double>`, `List<AddingMachine>` all from one `List.java`

<T> in the class/interface/method declaration says that the type will be defined (at compile time) by the code that uses a class or method

Callers must specify `<String>`, `<ComputeResult>`, etc when they create a variable of that type.

Advanced usage: you can use `?` to allow any type or one of several types; ex

```
List<?> whoCaresWhatsInTheList;
```



Generics

Generics have some rough edges because they were added in a backwards-compatible way; in particular, the `<>` type information **only** exists at compile time, not at runtime, and it is optional even there.

If you write

```
List<String> strings = new ArrayList<String>();  
strings.add("This isn't an integer");  
List noTypeSafetyList = strings;  
List<Integer> hahaAllTheBugs = noTypeSafetyList;  
int x = hahaAllTheBugs.get(0);
```

it will compile. If you try to run it, it will **crash** on the last line with a `ClassCastException`

Less obvious examples will break in a **hard to track down** way when you run the code by throwing `ClassCastException`s that look like they should be prevented by the Java language spec

⇒ **Avoid** these bugs by always specifying a type for any class or interface that is generic. If you see a warning about **raw types**, you need to add a `<>` type definition to your variable. `<?>` counts!

Generics (Beyond the Basics)

Very expressive framework

Fills in the missing gap with "ability" interfaces (Closeable, Identifiable, etc).

Ex: A method that takes in a List of Identifiables, and returns a Map from id to object

```
public interface Identifiable {  
    long getId();  
}
```

```
public Map<Long, Identifiable> getIdMap(List<Identifiable> data);
```

Generics Example

Problem 1: Using the method

The naive approach doesn't compile!

While DataClass is a subtype of Identifiable,

List<DataClass> is **not a subtype** of List<Identifiable> (for good reason - what could break if it were?)

```
public void processSomeData(List<DataClass> data) {  
    Map<Long, Identifiable> idMap = getIdMap(data);  
}
```

```
static class DataClass implements Identifiable {  
    private final String data;  
  
    public DataClass(String data) {  
        this.data = data;  
    }  
  
    @Override  
    public long getId() {  
        return data.hashCode();  
    }  
}
```

Generics Example

Solution: Use the **extends** keyword to define type characteristics

```
public Map<Long, Identifiable> getIdMap(List<? extends Identifiable> data);
```

Generics Example

Problem 2: Using the return value

Now it compiles, but getting data from the map requires a **cast**

```
static class DataClass implements Identifiable {  
    private final String data;  
  
    public DataClass(String data) {  
        this.data = data;  
    }  
  
    @Override  
    public long getId() {  
        return data.hashCode();  
    }  
}
```

```
public void processSomeData(List<DataClass> data) {  
    Map<Long, Identifiable> idMap = getIdMap(data);  
    Identifiable result = idMap.get(1);  
    String info = ((DataClass)result).data;  
}
```

Generics Example

Solution: Define a type variable to force type safety between parameters and output

```
public <T extends Identifiable> Map<Long, T> getIdMap(List<T> data);
```

Generics Example

Now the client code looks good!

```
static class DataClass implements Identifiable {  
    private final String data;  
  
    public DataClass(String data) {  
        this.data = data;  
    }  
  
    @Override  
    public long getId() {  
        return data.hashCode();  
    }  
}
```

```
public void processSomeData(List<DataClass> data) {  
    Map<Long, DataClass> idMap = getIdMap(data);  
    DataClass result = idMap.get(1);  
    String info = result.data;  
}
```

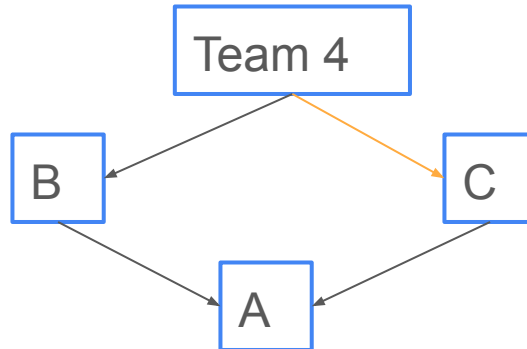
Creating Easily-Pluggable Libraries

Team/Company #1 creates Library A with some useful interfaces

Team/Company #2 builds on top of Library A with Library B

Team/Company #3 also builds on top of Library A with Library C

Team/Company #4 is using Library B - how can Team #3 make Library C useful in this situation?



Generics and Multi-Interface

Ex: Fully reading data from some closeable, identifiable stream

Team 3:

```
public static <T extends Identifiable & Closeable & Iterable<String>>
    List<String> readFully(T inputStream) {

    System.out.println("Reading stream: " + inputStream.getId());

    List<String> result = new ArrayList<>();

    try (inputStream) {
        for (String s : inputStream) {
            result.add(s);
        }
    } catch (IOException e) {
        System.err.println("Error reading stream: " + inputStream.getId());
    }
    return result;
}
```

Generics and Multi-Interface

Team 2:

```
static class CustomInputStream implements Identifiable, Closeable, Iterable<String> {  
    private final List<Integer> data;  
  
    public CustomInputStream(List<Integer> numbers) {  
        this.data = numbers;  
    }  
  
    @Override  
    public long getId() {  
        return data.hashCode();  
    }  
  
    @Override  
    public Iterator<String> iterator() {  
        return data.stream().map(number -> "" + number).iterator();  
    }  
  
    @Override  
    public void close() throws IOException {  
        // nothing to do  
    }  
}
```

Generics and Multi-Interface

Team 4:

```
public void processSomeData(CustomInputStream stream) {  
    System.out.println(readFully(stream));  
}
```

Your Turn!

1. Write a method that computes the length of an Iterable - make it as generic as possible
2. Add the following interface to your project:

```
public interface Sizeable {  
    public int getSize();  
}
```

3. Modify the method to require that the Iterable contain items that are Sizeable, and now compute the total size of everything in the Iterable
4. Verify that your method is easy to use: create a class that implements Sizeable, and check that you can pass a List of that type to your method from (3) (**Note:** List<T> implements Iterable<T>)

Solution

```
public class GenericIteratorCounter {  
  
    public int countSize(Iterable<? extends Sizeable> input) {  
        int totalSize = 0;  
        for (Sizeable value : input) {  
            totalSize += value.getSize();  
        }  
        return totalSize;  
    }  
  
    public static void main(String[] args) {  
        List<SizeableType> listOfLists = new ArrayList<>();  
        new GenericIteratorCounter().countSize(listOfLists);  
    }  
  
    public static class SizeableType implements Sizeable {  
        List<String> list = new ArrayList<>();  
        @Override  
        public int getSize() {  
            return list.size();  
        }  
    }  
}
```

Let's Review

Exceptions:

Three different ways to signal errors to a user: Exception, Error Codes/Return Values, Optional

- What each is and when to use them

try/catch/finally - syntax and what each block does

try-with-resources - when to use it, why to use it

Generics:

- syntax, including ? and ? extends
- How the compiler uses this to enforce type safety
- When should you add a type parameter
- How to use code that has a type parameter